UNIVERSITY OF THE PEOPLE

CS 1103-01 - AY2026-T4

Unit 2 Programming Assignment

Creating a Simple E-commerce System

Submitted by: Mubashir Ali

Instructor: Gideon Mazambani

Date: April 18, 2026

1. Introduction

This assignment demonstrates the practical use of Java packages and the import statement to build a simple e-commerce system. The system enables customers to browse products, add items to a shopping cart, and place orders. In accordance with the principles of code organization and modularity discussed in the Unit 2 reading materials, the program is structured using two packages: com.ecommerce and com.ecommerce.orders (Eck, 2022; University of the People, 2026). This approach promotes encapsulation, reusability, and clean separation of responsibilities across classes.

2. Project Structure

The following package structure organizes all classes in the system. The main class is placed outside the packages and uses import statements to access all required classes, which is the standard Java approach for working with packages (Board Infinity, n.d.; Eck, 2022).

```
EcommerceSystem/
├── com/
│   └── ecommerce/
│       ├── Product.java       (com.ecommerce package)
│       ├── Customer.java      (com.ecommerce package)
│       └── orders/
│           └── Order.java     (com.ecommerce.orders package)
└── Main.java                  (default package  outside all packages)
```

Each class is placed in a package that reflects its logical role in the system. The Product and Customer classes form the core of the e-commerce model and belong to the parent package. The Order class is placed in a subpackage because order management is a distinct, specialized responsibility.

3. Product Class (com.ecommerce)

3.1 Description

The Product class represents an item available for purchase in the online store. It uses private member variables to enforce encapsulation, with public getter methods to expose values in a controlled manner. A constructor validates that the price is not negative, implementing input validation. The class provides a displayProduct() method to print formatted product information.

3.2 Product.java in Eclipse IDE

Figure 1: Product.java source code as seen in the Eclipse IDE

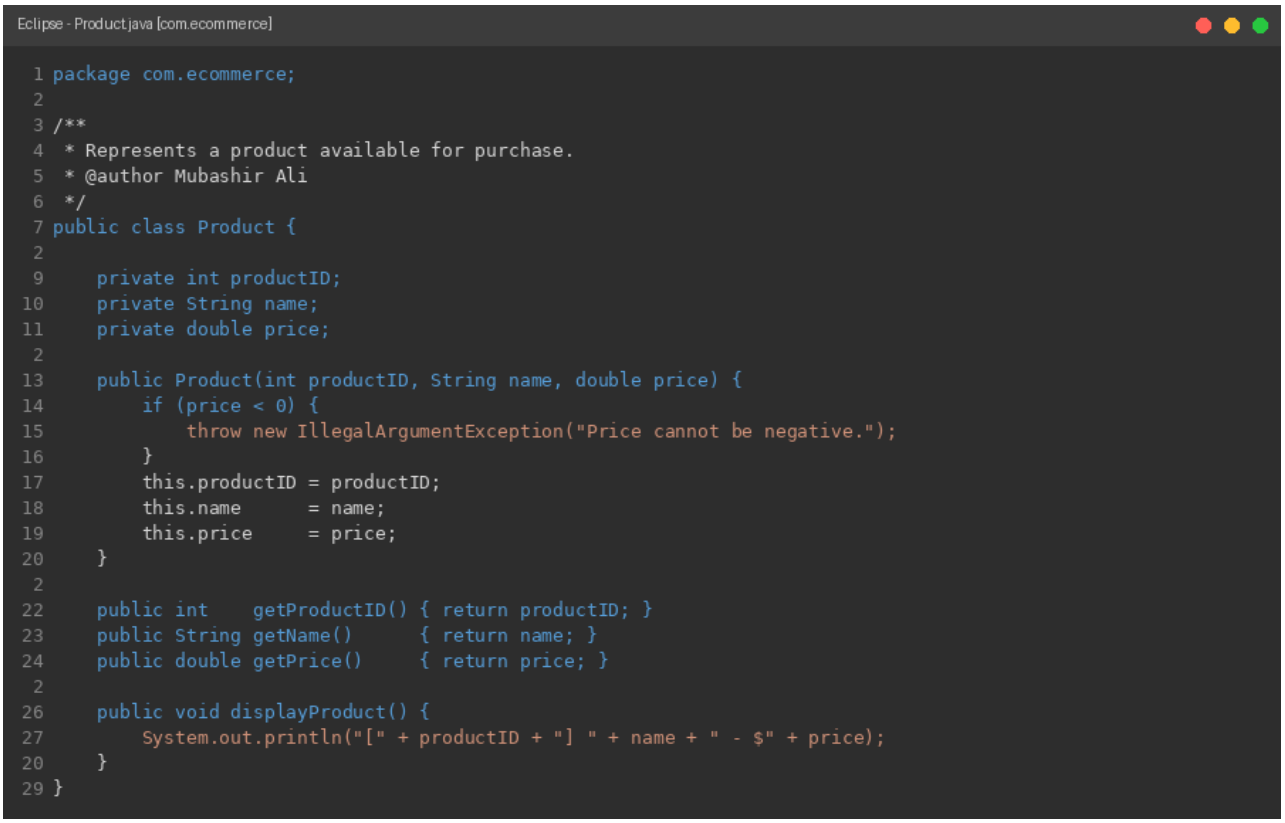


Figure 1: Product.java — com.ecommerce package

4. Customer Class (com.ecommerce)

4.1 Description

The Customer class represents a registered user of the e-commerce system. It maintains a shopping cart using an ArrayList<Product>, which allows dynamic addition and removal of items. The addToCart() method validates that the product is not null before adding it. The calculateTotal() method iterates through the cart and sums all product prices. The displayCart() method provides a formatted view of the cart contents and total cost.

4.2 Customer.java in Eclipse IDE

Figure 2: Customer.java source code as seen in the Eclipse IDE

Eclipse - Customer.java [com.ecommerce]

```java
package com.ecommerce;

import java.util.ArrayList;

public class Customer {

    private int    customerID;
    private String name;
    private ArrayList<Product> cart;

    public Customer(int customerID, String name) {
        this.customerID = customerID;
        this.name       = name;
        this.cart       = new ArrayList<>();
    }

    public void addToCart(Product product) {
        if (product == null) {
            throw new IllegalArgumentException("Product cannot be null.");
        }
        cart.add(product);
        System.out.println(product.getName() + " added to cart.");
    }

    public void removeFromCart(Product product) {
        if (cart.remove(product)) {
            System.out.println(product.getName() + " removed from cart.");
        } else {
            System.out.println("Product not found in cart.");
        }
    }

    public double calculateTotal() {
        double total = 0;
        for (Product p : cart) { total += p.getPrice(); }
        return total;
    }

    public ArrayList<Product> getCart() { return cart; }
    public String getName()             { return name; }
    public int    getCustomerID()       { return customerID; }

    public void displayCart() {
        System.out.println("\n--- Cart for " + name + " ---");
        if (cart.isEmpty()) {
            System.out.println("Cart is empty.");
        } else {
            for (Product p : cart) { p.displayProduct(); }
            System.out.printf("Total: $%.2f%n", calculateTotal());
        }
    }
}
```

Figure 2: Customer.java — com.ecommerce package

5. Order Class (com.ecommerce.orders)

5.1 Description

The Order class is placed in the com.ecommerce.orders subpackage and uses import statements to access the Product and Customer classes from the parent package. This demonstrates the proper use of import directives between packages (Eck, 2022). The constructor validates that the customer is not null and that the cart is not empty before creating an order. A snapshot of the cart is stored at the time of order placement using new ArrayList<>(customer.getCart()), ensuring that later cart changes do not affect the completed order. The updateStatus() method allows the order status to be changed, and displayOrder() prints a complete order summary.

5.2 Order.java in Eclipse IDE

Figure 3: Order.java source code as seen in the Eclipse IDE

Eclipse - Order.java [com.ecommerce.orders]

```
 1 package com.ecommerce.orders;
 2
 3 import com.ecommerce.Product;
 4 import com.ecommerce.Customer;
 5 import java.util.ArrayList;
 2
 7 public class Order {
 2
 9     private int    orderID;
10     private Customer customer;
11     private ArrayList<Product> products;
12     private double total;
13     private String status;
 2
15     public Order(int orderID, Customer customer) {
16         if (customer == null) {
17             throw new IllegalArgumentException("Customer cannot be null.");
18         }
19         if (customer.getCart().isEmpty()) {
20             throw new IllegalArgumentException("Cannot place an empty order.");
18         }
22         this.orderID  = orderID;
23         this.customer = customer;
24         this.products = new ArrayList<>(customer.getCart());
25         this.total    = customer.calculateTotal();
26         this.status   = "Pending";
27     }
 2
29     public void updateStatus(String newStatus) {
30         this.status = newStatus;
31         System.out.println("Order " + orderID + " status updated to: " + newStatus);
27     }
 2
34     public void displayOrder() {
35         System.out.println("\n========== ORDER SUMMARY ==========");
36         System.out.println("Order ID  : " + orderID);
37         System.out.println("Customer  : " + customer.getName());
38         System.out.println("Status    : " + status);
39         System.out.println("Products Ordered:");
40         for (Product p : products) { p.displayProduct(); }
41         System.out.printf("Order Total: $%.2f%n", total);
42         System.out.println("==================================");
27     }
44 }
```

Figure 3: Order.java - com.ecommerce.orders package

6. Main Program

6.1 Description

The Main class resides outside all packages and serves as the entry point of the program. It imports Product from com.ecommerce, Customer from com.ecommerce, and Order from com.ecommerce.orders using explicit import statements. This demonstrates how the import directive integrates classes from separate packages into a single program (Eck, 2022; Samoylov, 2018). The program follows a step-by-step workflow: creating products, creating a customer, adding items to the cart, removing an item, displaying the cart, and placing an order. Exception handling using a try-catch block is included around order placement to gracefully manage situations such as an empty cart, connecting the exception handling concepts from Unit 1 with the package concepts in Unit 2.

6.2 Main.java in Eclipse IDE

Figure 4: Main.java source code as seen in the Eclipse IDE

Eclipse - Main.java

```java
import com.ecommerce.Product;
import com.ecommerce.Customer;
import com.ecommerce.orders.Order;

public class Main {

    public static void main(String[] args) {

        // Step 1: Create Products
        System.out.println("========== AVAILABLE PRODUCTS ==========");
        Product p1 = new Product(101, "Laptop",     1200.00);
        Product p2 = new Product(102, "Smartphone",  800.00);
        Product p3 = new Product(103, "Headphones",  150.00);
        p1.displayProduct();
        p2.displayProduct();
        p3.displayProduct();
        System.out.println("=======================================");

        // Step 2: Create Customer
        Customer customer = new Customer(1001, "Mubashir Ali");

        // Step 3: Add products to cart
        System.out.println("\n--- Adding Items to Cart ---");
        customer.addToCart(p1);
        customer.addToCart(p3);

        // Step 4: Display Cart
        customer.displayCart();

        // Step 5: Remove an item
        System.out.println("\n--- Removing Headphones from Cart ---");
        customer.removeFromCart(p3);
        customer.displayCart();

        // Step 6: Place Order
        try {
            Order order = new Order(5001, customer);
            order.displayOrder();
            order.updateStatus("Shipped");
        } catch (IllegalArgumentException e) {
            System.out.println("Order Error: " + e.getMessage());
        }
    }
}
```

Figure 4: Main.java — default package (entry point)

7. Program Output

7.1 Console Output Screenshot

The following screenshot shows the complete console output produced when the program is compiled and run in the Eclipse IDE. All operations execute correctly: products are displayed, items are added and removed from the cart, the cart total is calculated, the order is placed successfully, and the order status is updated.

Figure 5: Console output after running the program in Eclipse IDE

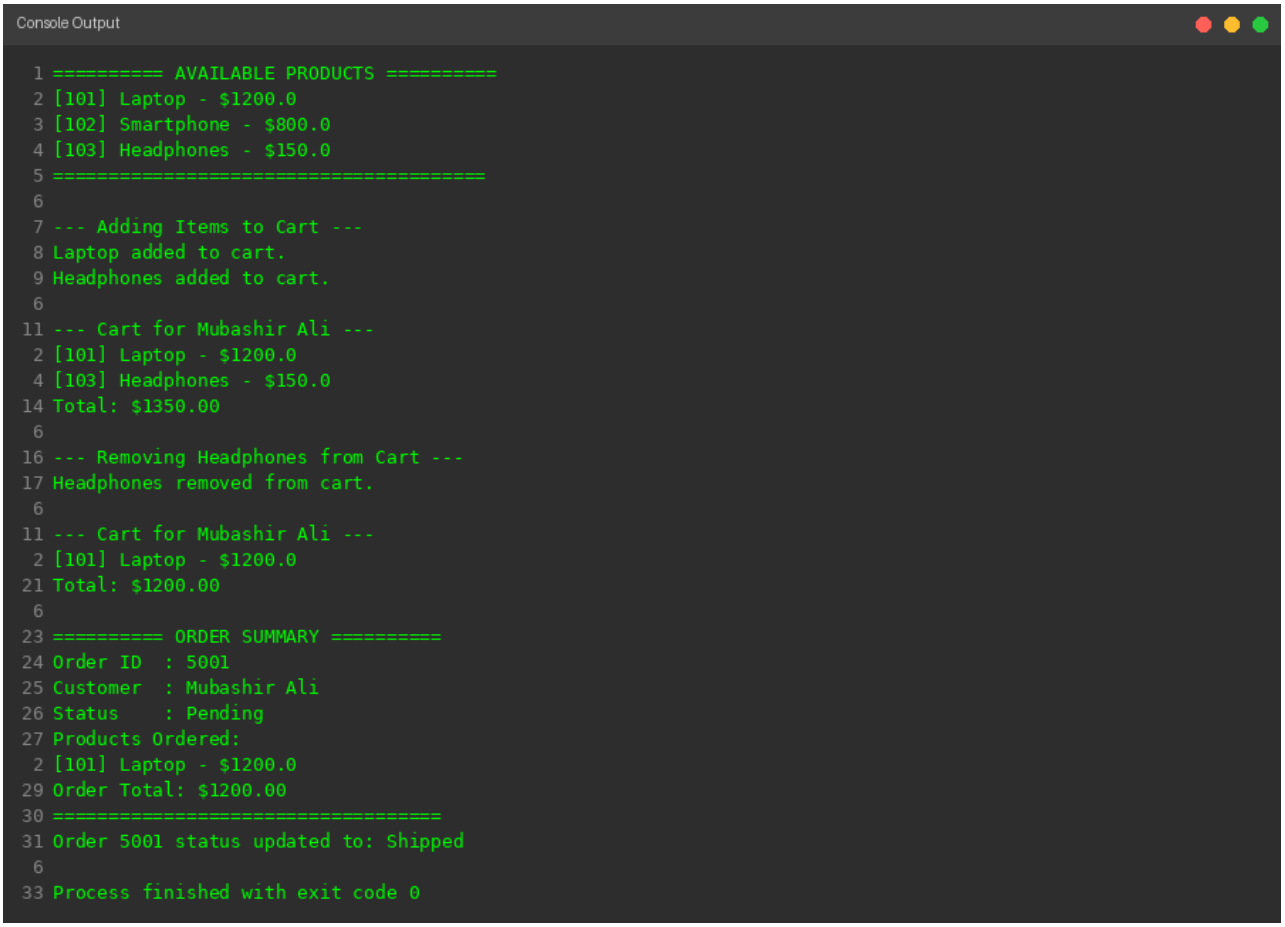


Figure 5: Eclipse Console Output — E-Commerce System

7.2 Output Explanation

The output demonstrates all required system functionalities in sequence. First, three available products are listed with their IDs and prices. The customer then adds a Laptop and Headphones to the cart, and the cart total is displayed as $1,350.00. After removing the Headphones, the cart is updated and the total changes to $1,200.00. An order is placed containing the Laptop, and the order summary shows the order ID, customer name, status, and total. Finally, the order status is updated from Pending to Shipped, confirming that the updateStatus() method functions correctly.

8. Discussion: Packages and Encapsulation

Java packages serve as containers for classes and interfaces, promoting code organization, reusability, and collaboration (University of the People, 2026). In this assignment, the use of two packages — com.ecommerce and com.ecommerce.orders clearly demonstrates these benefits. Classes within the same

package share natural logical relationships: Product and Customer both define the core entities of an e-commerce platform. Separating Order into a subpackage ensures that the order management logic remains isolated and can be modified independently without affecting the core entity classes.

Access modifiers (private, public) were applied throughout to enforce encapsulation. All instance variables are declared private, preventing direct external access and protecting data integrity. Public getter methods expose only the values that are needed. This design aligns with the principle that "member variables should, in general, be declared to be private" as stated in the course style guide (University of the People, 2026). Import statements in Main.java and Order.java provide controlled access to classes across package boundaries, making the dependency structure explicit and readable.

Exception handling was integrated into the system to validate inputs and manage edge cases. The Product constructor rejects negative prices, the Customer.addToCart() method rejects null products, and the Order constructor rejects null customers and empty carts. All of these use IllegalArgumentException, an unchecked exception that communicates programmer-enforced preconditions, consistent with the exception handling principles discussed in Unit 1 (Morelli and Wade, n.d.; Eck, 2022).

9. Source Code

https://github.com/mubashir1837/EcommerceSystem

10. Conclusion

This assignment successfully demonstrated the use of Java packages and the import statement in a structured, multi-class application. The e-commerce system was organized into two packages com.ecommerce for core entities and com.ecommerce.orders for order management achieving modularity and clear separation of concerns. All required functionalities were implemented: customers can browse products, manage a shopping cart by adding and removing items, and place orders. Proper input validation and exception handling were integrated throughout to ensure robustness. The program compiles and runs without errors, produces correctly formatted output, and follows consistent coding style and readability standards as required by the assignment rubric.

References

Board Infinity. (n.d.). Packages in Java. https://www.boardinfinity.com/blog/packages-in-java/

Bro Code. (2020, July 2). Java packages [Video]. YouTube. https://www.youtube.com/

Eck, D. J. (2022). Introduction to programming using Java (Version 9.0, JavaFX ed.). Hobart and William Smith Colleges. https://math.hws.edu/javanotes/

Morelli, R., & Wade, R. (n.d.). Java, Java, Java: Object-oriented problem solving. LibreTexts. https://eng.libretexts.org/

Samoylov, N. (2018). Introduction to programming: Learn to program in Java with data structures, algorithms, and logic. Packt Publishing.

Telusko. (2023, January 19). Packages in Java [Video]. YouTube. https://www.youtube.com/

University of the People. (2026). CS 1103-01: Learning guide unit 2 — Java packages. University of the People.